Implicit indentation

One of the things that you've just seen is that the rewriter adds implicit indentation. For example, when you write a function inline, the rewriter assumes that you meant it *with* indentation:

```
I = (x) -> x
```

The `[INDENT]` in the token stream is placed before the function body:

```
I = (x) ->[INDENT]x
```

Notice that indentation doesn't require a newline. Of course, in practice you always *think* of indentation as being accompanied by a preceding newline because that's the only way you can write it.

Newlines

The rewriter also removes some leading and mid-expression newlines:

```
a = 1
b = 2
```

Any important newline, such as one that indicates the end of an expression (a terminator) is preserved:



Further rewrites are performed for braces and parentheses.

Braces and parentheses

In most places, parentheses and braces are optional. This is convenient but can leave room for some ambiguity. The simplest way for the compiler to deal with these issues is with the rewriter: